

Database Management System : Suggestions and PYQ

PYQ-2024

Group-A :

1. Why do we need normalization?

To remove

- a. Redundancy in a DB
- b. Anomalies in a DB

2. Define participation constraints ?

Participation constraint specifies whether the participation of an entity set in a relationship set is mandatory or optional.

It has two types:

- A. Total participation : Every entity in a entity set must participate to atleast one relationship instance.
- B. Partial participation: Some entity in a entity set may not participate in relationship.

3. Differentiate schema and instance.

Difference between Schema and Instance

<u>Schema</u>	<u>Instance</u>
Schema is the structure or design of the database.	Instance is the actual data stored in the database at a particular time.
It is defined using DDL commands .	It is modified using DML commands .
Schema changes rarely .	Instance changes frequently .
Schema is also called database blueprint .	Instance is also called database snapshot .

4. What are ACID Properties?

ACID properties ensure **reliable transaction processing** in a database.

- **A – Atomicity**: It is either execute all the operations of transaction or execute none of the operations.
- **C – Consistency**: The user requested operations to perform transaction must be logically correct.
- **I – Isolation**: Concurrent execution of 2 or more transactions must be equal to result of any serial execution of transaction.
- **D – Durability**: Once committed, changes are permanent even after failure.

5. What is a trigger?

A **trigger** is a database object that **automatically executes** a specified set of actions **when a certain event occurs** on a table or view (such as INSERT, UPDATE, or DELETE).

Example:

A trigger that updates stock quantity after a sale.

Group -B :

1. What is data independence ? Differentiate between physical and logical data independence.

Data independence is the ability of a **DBMS** to change the schema at one level **without affecting** the schema or programs at the next higher level.

Types of Data Independence:

1. **Physical Data Independence** – Changes in physical storage (file organization, indexing, storage devices) do **not affect** the logical schema.
2. **Logical Data Independence** – Changes in logical schema (tables, attributes, relationships) do **not affect** application programs or user views.

Difference between Physical and Logical Data Independence

Physical Data Independence

Concerned with **internal (physical)** level of database

Changes in storage structure do **not affect logical schema**

Examples: change in file organization, indexing, storage location

Easier to achieve

Mostly handled by DBMS automatically

Logical Data Independence

Concerned with **conceptual (logical)** level of database

Changes in logical schema do **not affect user views or programs**

Examples: add/remove attributes, split or merge tables

Harder to achieve

Requires careful database design

2. Explain about the following clauses with example – Group By and Order By and Aggregation functions.

1) GROUP BY Clause

GROUP BY is used to **group rows having the same values** in one or more columns. It is generally used with **aggregate functions**.

Example:

Find total salary department-wise.

```
SELECT dept_id, SUM(salary)
FROM Employee
GROUP BY dept_id;
```

2) ORDER BY Clause

ORDER BY is used to **sort the result set** in ascending (ASC) or descending (DESC) order.

Example:

Display employees sorted by salary (highest first).

```
SELECT name, salary  
FROM Employee  
ORDER BY salary DESC;
```

3) Aggregation Functions

Aggregate functions perform **calculations on a set of values** and return a single value.

Common aggregate functions:

- COUNT() – counts rows
- SUM() – total of values
- AVG() – average value
- MIN() – minimum value
- MAX() – maximum value

Example:

Find average salary of all employees.

```
SELECT AVG(salary)  
FROM Employee;
```

3. What is a weak entity type ? how to model it? Explain with suitable example.

Weak Entity Type

A **weak entity type** is an entity that **cannot be uniquely identified by its own attributes**.

It **depends on a strong (owner) entity** for its existence and identification.

Key points:

- Has **no primary key of its own**
 - Identified using a **partial key (discriminator)**
 - Existence depends on a **strong entity**
 - Has **total participation** in the identifying relationship
-

Modeling a Weak Entity

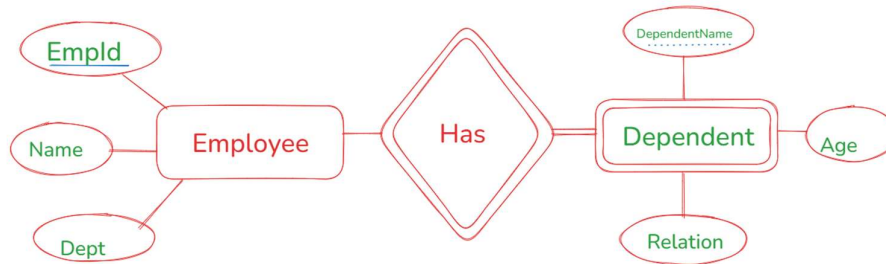
In ER diagram:

- Weak entity → **Double rectangle**
 - Identifying relationship → **Double diamond**
 - Partial key → **Dashed underline**
 - Relationship shows **total participation** (double line)
-

Example

Employee (strong entity) and Dependent (weak entity)

- **Employee:** EmpID (Primary Key), Name, Dept
- **Dependent:** DepName (Partial Key), Age, Relation
A dependent **cannot exist without an employee.**
Primary key of Dependent = (EmpID + DepName)



Explanation

- Dependent is a weak entity because DepName alone cannot uniquely identify it
- It is identified by combining **Employee's EmpID** and **Dependent's DepName**

4. Draw a ER diagram for hospital management system.

5. What is DML ? explain DML operations with example.

DML (Data Manipulation Language)

DML is a subset of SQL used to **retrieve, insert, update, and delete data** in database tables.

DML Operations

1. INSERT

Used to **add new records** into a table.

Example:

```
INSERT INTO Student VALUES (101, 'Ariyan', 'CSE');
```

2. SELECT

Used to **retrieve data** from a table.

Example:

```
SELECT * FROM Student;
```

(if SELECT Clause comes in exam, this is the ans)

3. UPDATE

Used to **modify existing records**.

Example:

```
UPDATE Student
SET branch = 'IT'
WHERE roll_no = 101;
```

4. DELETE

Used to **remove records** from a table.

Example:

```
DELETE FROM Student
WHERE roll_no = 101;
```

6. Distinguish between :

- a. Primary and secondary indexing
- b. Primary key and candidate key

a. Primary Indexing vs Secondary Indexing

<u>Primary Indexing</u>	<u>Secondary Indexing</u>
Created on the primary key	Created on non-primary key attributes
Data file is sorted on the indexed field	Data file is not necessarily sorted
Only one primary index per table	Multiple secondary indexes can exist
Index field has unique values	Index field may have duplicate values
Faster access	Slightly slower compared to primary index

b. Primary Key vs Candidate Key

<u>Primary Key</u>	<u>Candidate Key</u>
Candidate Key whose Field value cannot be NULL	Keys that can uniquely identify tuples
Only one primary key per table	Multiple candidate keys possible
Selected from candidate keys	One of the candidate keys is chosen as primary key
Used as main identifier	Potential identifiers

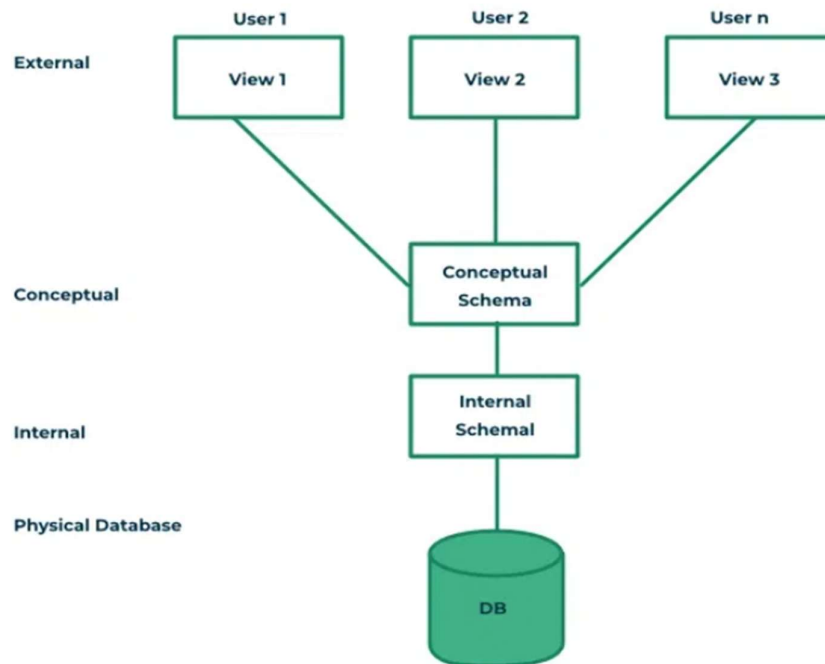
Group C

1.

- a. Draw and explain 3-tier schema architecture of database system.
- b. What is a Foreign Key constraint ?
- c. What is referential integrity ?
- d. Explain domain constraints and key constraints with example.

Ans:

a)



Three-Tier (Three-Schema) Architecture of DBMS

The **3-tier schema architecture** divides a database system into **three levels** to provide **data abstraction** and **data independence**.

1. External Level (View Level)

- Represents **user-specific views** of the database
- Different users can have **different views**
- Hides unnecessary data from users

Example: Student sees marks, admin sees full records

2. Conceptual Level (Logical Level)

- Represents the **overall logical structure** of the database
- Defines **entities, attributes, relationships, constraints**
- Independent of physical storage

3. Internal Level (Physical Level)

- Describes **how data is stored physically**
- Includes file organization, indexing, storage details

Advantages:

- Supports **logical and physical data independence**
- Improves **security and flexibility**

b) A foreign key constraint is used to link two tables.

Example:

Student(SID, Name, DeptID)

Department(DeptID)

DeptID in Student → Foreign Key

c) **Referential integrity** ensures that **relationships between tables remain valid.**

It means a foreign key value must:

- Match an existing primary key value in the referenced table, or
- Be NULL

d) 1. **Domain Constraints**→**restrict values**

Domain constraints define the **allowed set of values** for an attribute.

Example:

Age INT CHECK (Age >= 18)

Gender CHAR(1) CHECK (Gender IN ('M','F'))

Here, Age cannot be less than 18.

2. **Key Constraints**→ **ensures uniqueness**

Key constraints ensure **uniqueness of tuples** in a table.

Types: Primary key, Candidate key, Unique key

Example:

StudentID INT PRIMARY KEY

Email VARCHAR(50) UNIQUE

- *StudentID uniquely identifies each record*
- *Email cannot have duplicate values*

2.

- a. Explain insertion , deletion and update anomalies.
- b. Differentiate full Functional Dependency(FD) and partial FD.
- c. By means of an example show how BCNF is more stronger than 3 NF.
- d. Given a relation R(A , B , C , D) and FD ={ AB→CD , B→ C} , determine whether the given R is in 2 NF and if not convert it into 2 NF .

Ans : a)

Insertion Anomaly:

Occurs when **new data cannot be inserted** without adding unrelated data.

Example: Cannot add a new department unless at least one employee exists.

Deletion Anomaly:

Occurs when **deleting data unintentionally removes useful information.**

Example: Deleting the last employee of a department deletes department details.

Update Anomaly:

Occurs when **same data appears in multiple rows** and all copies are not updated consistently.

Example: Updating department name in one row but not others causes inconsistency.

b)

Difference between Full FD and Partial FD

<u>Full Functional Dependency</u>	<u>Partial Functional Dependency</u>
Non-key attribute depends on entire composite key	Non-key attribute depends on part of composite key
Occurs in 2NF or higher	Violates Second Normal Form (2NF)
Example: (Roll, Subject) → Marks	Example: (Roll, Subject) → StudentName (depends only on Roll)

c)

BCNF is stronger than 3NF because every BCNF relation is in 3NF, but not every 3NF relation is in BCNF.

Example

Consider a relation:

R(A, B, C)

Functional Dependencies:

- **$AB \rightarrow C$**
- **$C \rightarrow B$**

Step 1: Find candidate keys

- **$AB^+ = ABC$** , so **AB** is a candidate key.
- **$AC^+ = ABC$** , so **AC** is also a candidate key.

Prime attributes: A, B, C (all are part of some candidate key)

Check for 3NF

For each FD:

1. **$AB \rightarrow C$**
 - AB is a super key
2. **$C \rightarrow B$**
 - C is **not** a super key
 - But **B is a prime attribute**

Therefore, the relation is in **3NF**.

Check for BCNF

BCNF requires:

For every FD $X \rightarrow Y$, **X must be a super key**.

- **$C \rightarrow B$** violates BCNF because **C is not a super key**.

Therefore, the relation is **not in BCNF**.

Conclusion

- The relation **satisfies 3NF**
- The relation **violates BCNF**

This proves that **BCNF is stricter (stronger) than 3NF**.

d)

3. Consider the SAILOR Database

Sailors(sid:string , sname: string , rating :integer, age: real)

Boats(bid:integer , bname : string,colot :string)

Reserves (sid:integer , bid: integer , day: date)

Based on the above schema , answer the queries in RA

- Find the color of boats reserved by 'Lubber'.
- Find the names of the sailors who have reserved atleast one boat.
- Find the name of the sailors who have reserved either a red or green boat.
- Find the name of the sailors who have reserved both a green and a red boat.
- Find names of sailors who have reserved all boats.

4. Differentiate between serial and serializable schedule.

Explain the problem that would arise when concurrency control is not provided by the database system.

Discuss about conflict serializability with an example.

<i>Serial Schedule</i>	<i>Serializable Schedule</i>
<i>Transactions execute one after another with no overlap</i>	Transactions overlap in time
<i>Simple and safe</i>	More efficient
<i>No concurrency</i>	Concurrency allowed
<i>Always consistent</i>	Consistent if equivalent to some serial schedule

If concurrency control is absent, the following problems may occur:

1. Lost Update : (x=10)

- Two transactions update the same data.
- One update overwrites the other.
- Result: incorrect final value.

2. Dirty Read:

- A transaction reads data written by another **uncommitted** transaction.
- If the transaction rolls back → inconsistency.

3. Unrepeatable Read

- A transaction reads the same data twice and gets **different values**
- Due to another transaction updating it in between.

4. Inconsistent Retrieval (Incorrect Summary)

- One transaction reads part of the database while another is updating it.
- Result: partial and inconsistent data.

A schedule is **conflict serializable** if it can be transformed into a **serial schedule** by swapping **non-conflicting operations**.

Conflicting operations:

- Belong to **different transactions**
- Access the **same data item**

- At least one operation is a **write**

5. Write short notes on

- a. Recoverable schedule
- b. Data redundancy and concurrency
- c. Multilevel indexing
- d. Lossless join

a. Recoverable Schedule:

A **recoverable schedule** is one in which a transaction **commits only after the transactions whose data it has read have committed**.

This ensures that if a transaction fails, the system can **roll back dependent transactions** and maintain database consistency.

Recoverable schedules are important for **safe recovery** after system failures.

b. Data Redundancy and Concurrency

Data Redundancy

Data redundancy refers to **duplication of data** in the database.

It leads to **waste of storage space** and may cause **data inconsistency** during update, insertion, or deletion.

Concurrency

Concurrency means **simultaneous execution of multiple transactions**.

It improves **system performance and throughput**, but requires **concurrency control mechanisms** to maintain data consistency.

c. Multilevel Indexing

Multilevel indexing is used when a **single-level index becomes too large** to fit in memory.

In this method, an **index is built on another index**, forming multiple levels.

It **reduces the number of disk accesses** and speeds up data retrieval.

Examples include **B-tree** and **B⁺-tree indexing**.

d. Lossless Join

A decomposition of a relation is said to have a **lossless join** if the **original relation can be perfectly reconstructed** by joining the decomposed relations.

It ensures that **no information is lost** and **no spurious tuples** are generated.

The condition for lossless join is:

$(R_1 \cap R_2 \rightarrow R_1)$ or $(R_1 \cap R_2 \rightarrow R_2)$.

PYQ – 2023

Grp B

1. What are Armstrong axioms ? Prove the sixth inference rule.

Armstrong's Axioms

Armstrong's axioms are a set of **sound and complete inference rules** used to derive all **functional dependencies (FDs)** in a relational database.

The three basic axioms are:

2. **Reflexivity Rule**
If $Y \subseteq X$, then
 $X \rightarrow Y$
3. **Augmentation Rule**
If $X \rightarrow Y$, then
 $XZ \rightarrow YZ$ for any Z
4. **Transitivity Rule**
If $X \rightarrow Y$ and $Y \rightarrow Z$, then
 $X \rightarrow Z$

Sixth Inference Rule (Union Rule)

Statement:

If

- $X \rightarrow Y$
- $X \rightarrow Z$

then

$X \rightarrow YZ$

Proof of Union Rule using Armstrong's Axioms

Given:

- $X \rightarrow Y \dots (1)$
- $X \rightarrow Z \dots (2)$

Step 1: Apply Augmentation on (1)

Augment both sides with **Z**:

- $XZ \rightarrow YZ$

Step 2: Apply Reflexivity

Since $X \subseteq XZ$,

- $XZ \rightarrow X$

Step 3: Apply Transitivity

From:

- $XZ \rightarrow X$
- $X \rightarrow Z$

We get:

- $XZ \rightarrow Z$

Step 4: Combine Results

From:

- $XZ \rightarrow YZ$
- $XZ \rightarrow Z$

It follows that:

- $X \rightarrow YZ$

Conclusion

The **Union Rule** can be derived from **Armstrong's axioms**, hence it is a **valid inference rule**.

5. Explain ACID properties of a transaction. (PYQ-24)

6. What is cascading rollback? What is an irrecoverable schedule.

Cascading Rollback

Cascading rollback occurs in a transaction schedule when the rollback (abort) of one transaction forces the rollback of other transactions that have read its **uncommitted (dirty) data**.

- If **T1** writes a data item **X** and **T2** reads **X** before **T1** commits, then **T2** depends on **T1**.

- If **T1** later aborts, **T2** must also be rolled back to maintain consistency.
- This chain effect (one rollback causing others) is called **cascading rollback**.
It is undesirable because it increases recovery time and wastes work.

Irrecoverable Schedule

An **irrecoverable schedule** is one in which a transaction **commits after reading data written by another transaction that later aborts**.

- Example: **T1** writes **X**, **T2** reads **X** and commits, then **T1** aborts.
- Since **T2** has already committed using invalid data, the system **cannot recover** the database to a consistent state.

Such schedules are **not allowed** in DBMS.

7. Explain with example 2 NF and 3 NF.

Second Normal Form (2NF)

A relation is in **Second Normal Form (2NF)** if:

1. It is already in **First Normal Form (1NF)** 2. **No partial dependency** exists

A relation is in **Third Normal Form (3NF)** if:

1. It is already in **2NF**
2. **No transitive dependency** exists

Grp C

1. Consider the table Emp(empid , name , sal , did) and DEPT(did , dname , dloc)
Write RA and SQL for each of the query.
 - a. Who earns the max salary in the 'sales' dept.
 - b. Which dept has the min number of employees
 - c. List the employee who earns more than the avg salary of his/her dept.
2. Define conflict(PYQ-24) and view serializability.
Justify with an example "every conflict ser. Is viewed as serializable but not vice-versa"

View Serializability

A schedule is **view serializable** if it is **view equivalent** to a serial schedule, meaning:

1. The same transaction reads the **initial value** of each data item
2. The same transaction reads the value written by the **same transaction**
3. The **final write** on each data item is done by the same transaction

3. What is 2PL?

What are its advantages ?

Explain the algorithm of time stamp based locking protocol.

Describe the preemptive and non-preemptive scheme of deadlock avoidance.

(a) Two Phase Locking (2PL) – 2.5 marks

Two Phase Locking (2PL) is a concurrency control protocol in which each transaction has **two distinct phases**:

1. Growing Phase

- Transaction can **acquire locks**
- Cannot release any lock

2. Shrinking Phase

- Transaction can **release locks**
- Cannot acquire any new lock

If all transactions follow 2PL, the schedule is **conflict serializable**.

(b) Advantages of 2PL – 2.5 marks

1. Guarantees **conflict serializability**
 2. Ensures **data consistency**
 3. Simple and widely used protocol
 4. Prevents lost update and dirty read problems
 5. Works well with lock-based DBMS systems
-

(c) Timestamp Based Locking (Timestamp Ordering Protocol) – Algorithm (5 marks)

In **timestamp ordering protocol**, each transaction T_i is assigned a unique timestamp $TS(T_i)$ when it starts.

For each data item X , DBMS maintains:

- $RTS(X) \rightarrow$ Read timestamp of X
 - $WTS(X) \rightarrow$ Write timestamp of X
-

Algorithm Rules

1. Read(X) by T_i

- If $TS(T_i) < WTS(X) \rightarrow$ Reject operation, **T_i is aborted**
- Else
 - Read X
 - Set $RTS(X) = \max(RTS(X), TS(T_i))$

2. Write(X) by T_i

- If $TS(T_i) < RTS(X) \rightarrow$ Abort T_i
- Else if $TS(T_i) < WTS(X) \rightarrow$ Abort T_i
- Else
 - Write X
 - Set $WTS(X) = TS(T_i)$

Properties

- Ensures **conflict serializability**
- **Deadlock free** (no waiting)
- May cause **starvation** due to repeated aborts

(d) Deadlock Avoidance Schemes (5 marks)

1. Preemptive Scheme (Wound–Wait)

- Based on **transaction timestamps**
- If **older transaction requests a lock held by younger**:
 - Older **wounds** younger $\rightarrow$ younger is aborted
- If **younger requests lock held by older**:
 - Younger **waits**

Characteristics:

- Preemptive (forces rollback)
 - Prevents deadlock
 - Older transactions favored
-

2. Non-Preemptive Scheme (Wait–Die)

- Also based on timestamps
- If **older transaction requests lock held by younger**:
 - Older **waits**
- If **younger requests lock held by older**:
 - Younger **dies** (aborted)

Characteristics:

- Non-preemptive
- Deadlock free
- Younger transactions may starve

4. Write the algo of finding Primary key and find out CK for the R(A , B , C D E F G H)
 $F = \{ A \rightarrow CD, AB \rightarrow EG, C \rightarrow G, D \rightarrow EF, AC \rightarrow D, G \rightarrow FH \}$

Write the algo of finding canonical cover of a FD set , find the canonical cover of
 $F = \{ W \rightarrow X, Y \rightarrow X, Z \rightarrow WXY, WY \rightarrow Z \}$

5. Explain dense and sparse indexing.
Discuss dynamic hashing technique.
Explain the steps of query optimization.

(a) Dense and Sparse Indexing (5 marks)

Dense Index

A **dense index** contains an **index entry for every search key value** (or every record).

Characteristics:

- Each index entry points to **one record**
- Faster search
- Requires **more storage**

Example:

If a table has 100 records, dense index has **100 index entries**

Advantage:

- Very fast access

Disadvantage:

- High space overhead
-

Sparse Index

A **sparse index** contains index entries for **some search key values only**, usually **one per data block**.

Characteristics:

- Index entry points to **first record of a block**
- Requires **sorted data file**
- Less storage used

Example:

If 100 records stored in 10 blocks, sparse index has **10 index entries**

Advantage:

- Saves space

Disadvantage:

- Slightly slower than dense index
-

Dense vs Sparse (1-line difference):

- Dense index → entry for **every record**
 - Sparse index → entry for **some records only**
-

(b) Dynamic Hashing Technique (5 marks)

Dynamic hashing allows the hash table to **grow and shrink dynamically** to avoid overflow and improve performance.

Extendible Hashing (Dynamic Hashing Method)

Main Components:

- **Hash function** produces binary hash value
 - **Directory** (array of pointers)
 - **Buckets**
 - **Global depth (GD)** and **Local depth (LD)**
-

Working:

1. Apply hash function to key
 2. Use first **GD bits** to index directory
 3. Directory points to a bucket
 4. If bucket overflows:
 - Split the bucket
 - Increase **local depth**
 - If needed, double directory size (increase GD)
-

Advantages:

- No long overflow chains
 - Efficient insertion and search
 - Adapts to file growth
-

Disadvantage:

- Directory consumes extra memory
-

(c) Steps of Query Optimization (5 marks)

Query Optimization selects the **most efficient execution plan** for a query.

Steps:

1. **Query Parsing & Translation**
 - SQL query is converted into **relational algebra expression**
2. **Query Decomposition**
 - Break query into smaller sub-queries
 - Remove redundancies
3. **Logical Optimization**
 - Apply equivalence rules
 - Push **selection** and **projection** operations early
4. **Physical Optimization**
 - Choose access paths (index / full scan)
 - Select join methods (nested loop, hash join)
5. **Cost Estimation & Plan Selection**
 - Estimate cost (I/O, CPU)
 - Choose **least-cost execution plan**

Some Important questions

1. What is Cardinality Constraint?

Answer:

Cardinality constraint defines the **maximum number of entities** that can participate in a relationship (1:1, 1:N, M:N).

2. Difference between Cardinality Constraint and Participation Constraint

Answer:

- **Cardinality** → Maximum participation
 - **Participation** → Minimum participation
-

3. What is an Identifying Relationship?

Answer:

A relationship that **links a weak entity to its owner entity** is called an identifying relationship.

4. What is Degree of a Relationship?

Answer:

The number of entity sets participating in a relationship is called the degree (Unary, Binary, Ternary).

5. What is Generalization?

Answer:

Generalization is a **bottom-up approach** of combining similar entities into a higher-level entity.

6. What is Relationship?

A **relationship** is an association between two or more entity sets in an ER model that shows how entities are connected to each other.

Example:

Employee **works in** Department.

Normalization - Assignment

1. What do you mean by Normalization? What are its advantages?

Normalization is the process of organizing data in a database to **reduce redundancy** and **avoid anomalies** by decomposing relations based on functional dependencies.

Advantages:

- Reduces data redundancy
- Eliminates insertion, deletion, and update anomalies
- Improves data consistency and integrity

- Makes database design efficient

2. Define

a. Prime attribute

An attribute that is part of **any candidate key** of a relation.

b. Super key

A set of one or more attributes that can **uniquely identify a tuple** in a relation.

c) Transitive Dependency

A dependency where a **non-key attribute depends on another non-key attribute** indirectly through a key.

d) Lossless Decomposition

A decomposition in which the **original relation can be reconstructed** without loss of information by joining the decomposed relations.

e) Dependency Preservation

A property of decomposition where **all functional dependencies can be enforced** without joining the relations.

f) Full and Partial Functional Dependency

- **Full FD:** A dependency where an attribute depends on the **entire candidate key**.
- **Partial FD:** A dependency where an attribute depends on **part of a composite key**.

3. Define 1 NF. What is Key attribute of a relation?

First Normal Form (1NF):

A relation is in **1NF** if all its attributes contain **atomic (indivisible) values** and there are no repeating groups.

Key Attribute:

An attribute that is **part of a candidate key** of a relation.

- Consider the relation
- $R(A, B, C, D, E)$
- with the following set of functional dependencies:
- $F = \{A \rightarrow BC, B \rightarrow D, D \rightarrow E\}$
- **(a) Find all the candidate keys of the relation R.**
- **(b) Determine the highest normal form of the given relation.**
- **(c) If the relation is not in Boyce–Codd Normal Form (BCNF), decompose it into relations that are in BCNF.**
- **(d) Justify that the decomposition is lossless.**

DDL (Data Definition Language) is used to **define, modify, and delete database structures** such as tables, schemas, and constraints.

List of DDL commands.

- CREATE
 - ALTER
 - DROP
 - TRUNCATE
 - RENAME
-

CREATE command with example.

The CREATE command is used to **create database objects**.

```
CREATE TABLE Student (  
    sid INT PRIMARY KEY,  
    name VARCHAR(20),  
    age INT  
);
```

Creates a table named Student.

ALTER command with example.

The ALTER command is used to **modify an existing table structure**.

```
ALTER TABLE Student ADD email VARCHAR(30);
```

Adds a new column email.

DROP command

The DROP command **removes a table permanently** along with its data.

```
DROP TABLE Student;
```

Log-based recovery is a technique that uses a **log file** to record all database modifications so that the system can **recover from failures**.